

"One of my big regrets is that Facebook hasn't had a major chance to shape the mobile operating system ecosystem."

- Mark Zuckerberg

CSE 312

Operating Systems

Spring 2026

Introduction to OS

© 2013-2026 Yakup Genç

1

Today

- Why, What, How, ... OS?

2

Disclaimer

Parts of these lecture slides contain work from many authors starting with A. S. Tanenbaum. I do not intend to provide a full list. However, I would like to emphasize that the slides are intended for the sole purpose of teaching of operating systems at Gebze Technical University. I also omit copyright notices of my own contributions. All the copyrighted material belong to their original owner(s).

Why Study Operating Systems

- OS are “magic”
 - most people don’t understand them – including system admins and computer scientists!
- OS are incredibly complex systems
 - “Hello, World” – program really 1 million lines of code
- Studying OS is learning how to deal with complexity
 - Abstractions (+interfaces)
 - Modularity (+structure)
 - Iteration (+learning from experience)

Why Study Operating Systems

- Understand an OS
 - Understand the inner workings of an OS
 - Enable you to write efficient/correct application code
- Learn to design an OS or other computer systems
- ...

Spring 2026

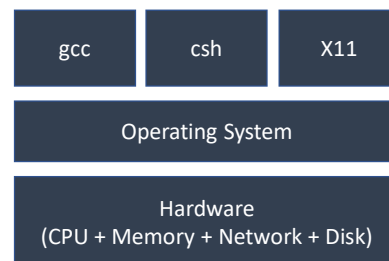
Operating Systems

5

5

What is an Operating System

- Software layer that sits between applications and hardware
- Performs services
 - Abstracts hardware
 - Provides protection
 - Manages resources



Spring 2026

Operating Systems

6

6

What is an Operating System

- Software that abstracts the computer hardware through interfaces
 - Hides the messy details of the underlying hardware
 - Presents users with a resource abstraction that is easy to use
 - Extends or virtualizes the underlying machine
- Manages the resources
 - Processors, memory, timers, disks, mice, network interfaces, printers, displays, ...
 - Allows multiple users and programs to share the resources and coordinates the sharing

Spring 2026

Operating Systems

7

7

OS as HW Abstraction (1)

- A modern computer consists of:
 - One or more processors
 - Main memory
 - Disks
 - Printers
 - Various input/output devices
- Managing all these components requires a layer of software – the operating system

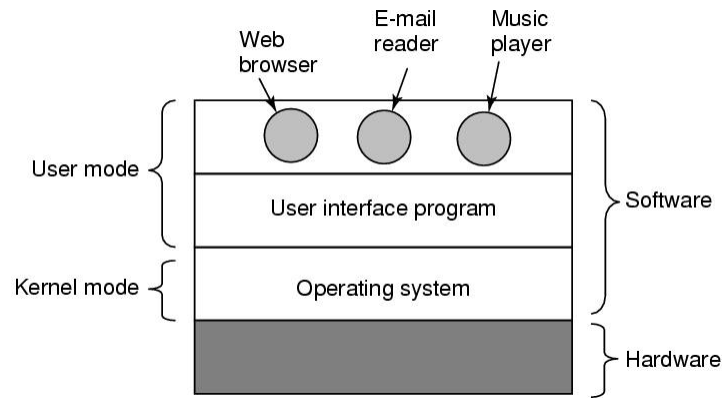
Spring 2026

Operating Systems

8

8

OS as HW Abstraction (2)



Where the operating system fits in...

Spring 2026

Operating Systems

9

9

What is an Operating System

- Software that abstracts the computer hardware through interfaces
 - Hides the messy details of the underlying hardware
 - Presents users with a resource abstraction that is easy to use
 - Extends or virtualizes the underlying machine
- **Manages the resources**
 - Processors, memory, timers, disks, mice, network interfaces, printers, displays, ...
 - Allows multiple users and programs to share the resources and coordinates the sharing

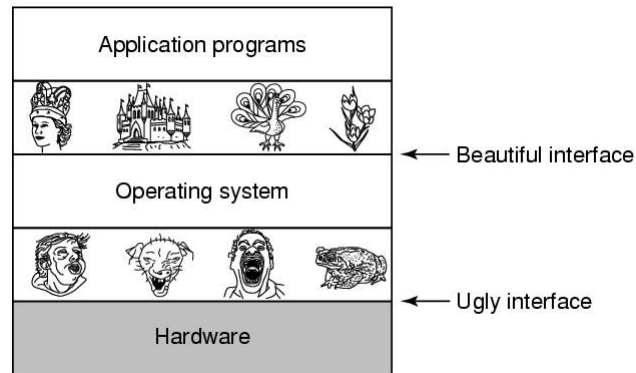
Spring 2026

Operating Systems

10

10

OS as Resource Manager (1)



Operating systems turn ugly hardware into beautiful abstractions.

Spring 2026

Operating Systems

11

11

OS as Resource Manager (2)

- Allow multiple programs to run at the same time
- Manage and protect memory, I/O devices, and other resources
- Includes multiplexing (sharing) resources in two different ways:
 - In time
 - In space

Spring 2026

Operating Systems

12

12

Operating System vs Kernel

- **Wide view**
 - Windows, Linux, Mac OSX are operating systems
 - Includes system programs, system libraries, servers, shells, GUI etc.
- **Narrow definition:**
 - OS often equated with the kernel
 - The Linux kernel; the Windows executive – the special piece of software that runs with special privileges and actually controls the machine
- In this class, usually mean the narrow definition
- For most people, though, the wide view is what counts

Spring 2026

Operating Systems

13

13

History of Operating Systems

- **Generations:**
 - (1945–55) Vacuum Tubes
 - (1955–65) Transistors and Batch Systems
 - (1965–1980) ICs and Multiprogramming
 - (1980–Present) Personal Computers

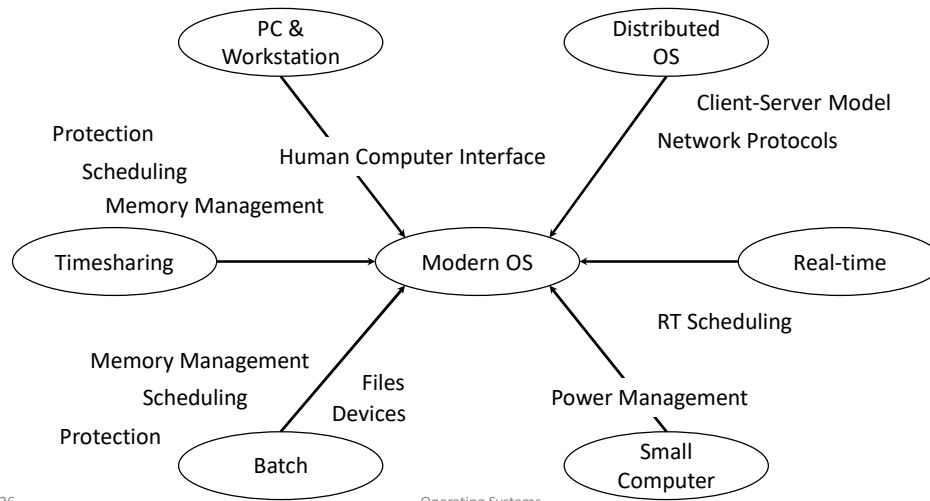
Spring 2026

Operating Systems

14

14

Evolution of Modern OS



Spring 2026

Operating Systems

15

15

Very Early OSs

- **Very Early Operating Systems**
 - The first computers had no operating systems
 - The concept isn't very meaningful on a plugboard or electromechanical device. . .
- **Dedicated time only, manually managed**

Spring 2026

Operating Systems

16

16

The Early Punchcard Era

- Punchcards — first used for data processing in the 1880s — made preparing programs easier
- But card readers were too slow; they tied up the (very expensive) CPU
- Printers had similar problems
- Solution: use slower, cheaper computer to copy input/output to/from mag tape

Spring 2026

Operating Systems

17

17

Computers Helping Themselves

- The realization that computers could help themselves was fairly early
- The first assembler was written by 1952
- Operating systems descend from this observation: that computers were useful for non-numeric and non-data processing work

Spring 2026

Operating Systems

18

18

Only a Partial Solution

- Even mag tapes were slower than the CPU
- Changing tapes is slow and manual (these weren't cassette tapes!)
- Something had to read the next "job" from the tape
- Enter the "monitor" program

Monitor Programs

- Monitors did a small part of running the computer
- (Human operators were still very busy)
- The operator's primary interaction was with the monitor, not the hardware

Primitive Monitors

- Often loaded from punch cards or tapes
- The same was true for most compilers — disks were rare (the first model, in 1956, had 50 two-foot platters, and held 5M bytes total. . .)
- Frequent need to reload monitor — no memory protection

Spring 2026

Operating Systems

21

21

I/O Libraries

- Monitors also provided I/O libraries
- I/O was complicated, especially if done efficiently
- Better to do it right once
- Examples: card column processing, disk rotational delay, card reader “clutch point”

Spring 2026

Operating Systems

22

22

Card Reader Programming

- Start the card moving
- Wait for a column interrupt
- When it occurs, read the value of that column
- Store it in the next memory location
- When card finishes, decide quickly if another card should be read

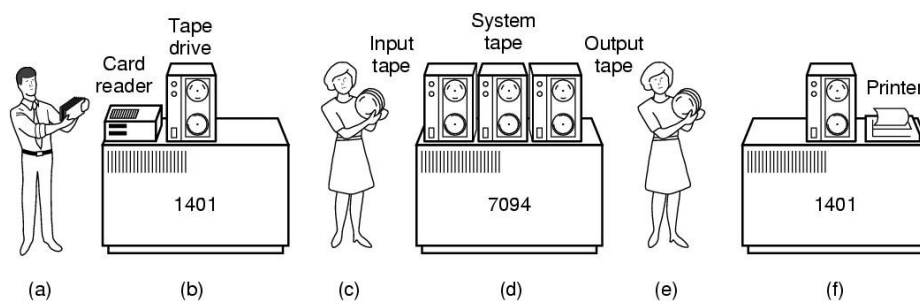
Spring 2026

Operating Systems

23

23

Transistors and Batch Systems (1)



An early batch system...

(a) Programmers bring cards to 1401.

(b) 1401 reads batch of jobs onto tape.

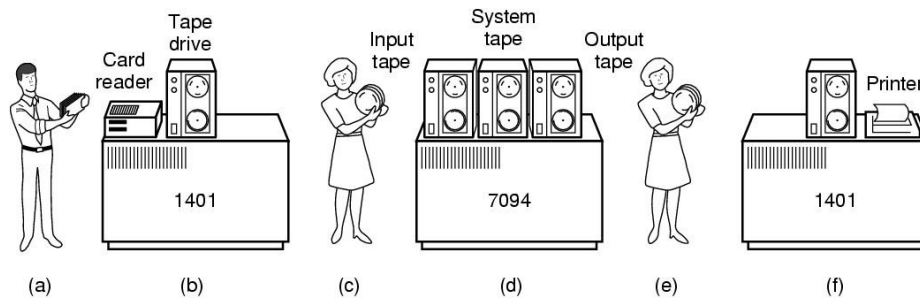
Spring 2026

Operating Systems

24

24

Transistors and Batch Systems (2)



An early batch system...

(c) Operator carries input tape to 7094.

(d) 7094 does computing.

(e) Operator carries output tape to 1401.

(f) 1401 prints output.

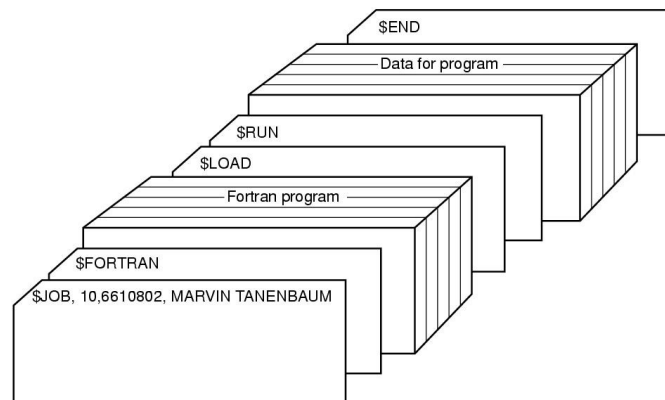
Spring 2026

Operating Systems

25

25

Transistors and Batch Systems (3)



Structure of a typical Fortran Monitor System job.

Spring 2026

Operating Systems

26

26

3rd Generation Computers – 1964

- Something recognizably an operating system
- Memory protection; multiprogramming
- Programs stored on disk
- Quote from a 1963 paper on disk usage:

Some thought has also been given to the storage of frequently used problem codes on the disk. There does not appear to be sufficient benefits attainable at the present time with the present problem “mix.” Also, the use of the disk for such activity is not attractive from a “computer-administrative” standpoint due to the addition and deletion of problems nor from the fact that less disk storage would be available for problem usage.
- On that monitor system, programmers allocated disk sectors themselves. . .

Spring 2026

Operating Systems

27

27

Why – Efficiency

- Mainframes still cost millions of dollars
- The CPU was idle while I/O was going on
- Save the cost of I/O computers

Spring 2026

Operating Systems

28

28

Why – Technology

- Important concepts: memory protection, virtual memory, interrupts
- Hardware had improved to the point that an OS was feasible:
 - Enough RAM (“core”) to hold it
 - Disks became the norm
 - Fast enough CPUs
- Beginnings of remote access

Spring 2026

Operating Systems

29

29

Multiprogramming

- Run several programs at once
- Protect each one’s memory against the others
- When one program needs to do I/O, let another program use the CPU
- As necessary, pre-empt a CPU-bound program to let another run
- Use OS “daemons” for spooling — Simultaneous Peripheral Output OnLine

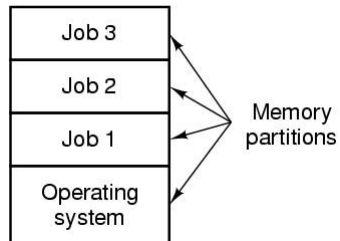
Spring 2026

Operating Systems

30

30

ICs and Multiprogramming



A multiprogramming system with three jobs in memory.

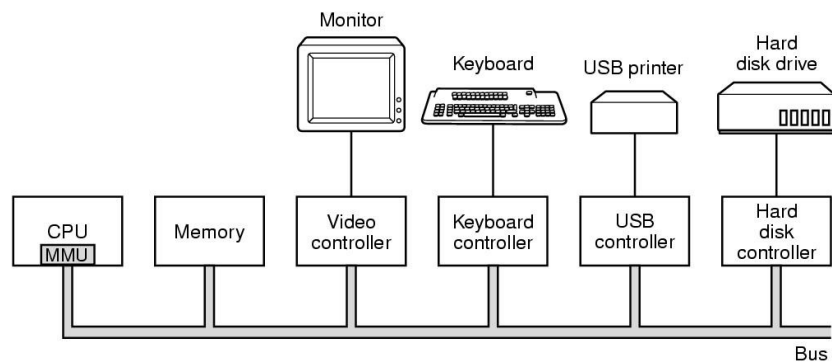
Spring 2026

Operating Systems

31

31

Computer Hardware Review



Some of the components of a simple personal computer.

Spring 2026

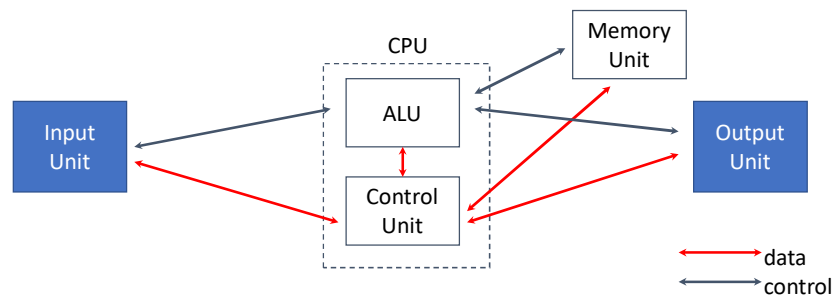
Operating Systems

32

32

Von Neumann Architecture

- Model of a computer that used stored programs
 - Both Data and Program stored in memory
 - Allows the computer to be “Re-programmed”
- CPU is central to the computer



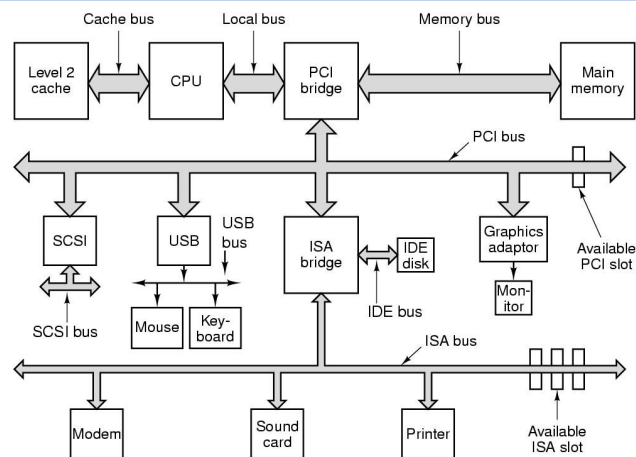
Spring 2026

Operating Systems

33

33

Von Neumann in Practice



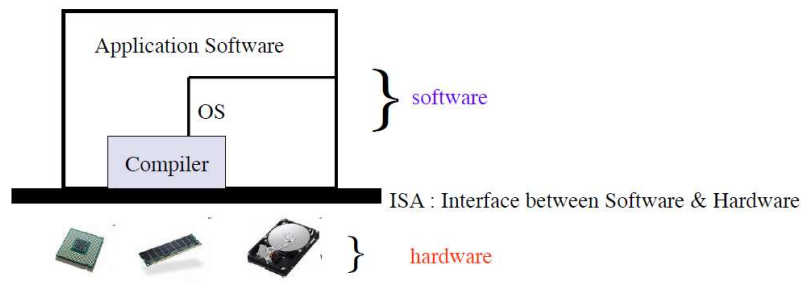
Spring 2026

Operating Systems

34

34

Instruction Set Architecture



Computer Architecture: ISA, memory, I/O, Power
Specialized H/W: GPU, co-processor

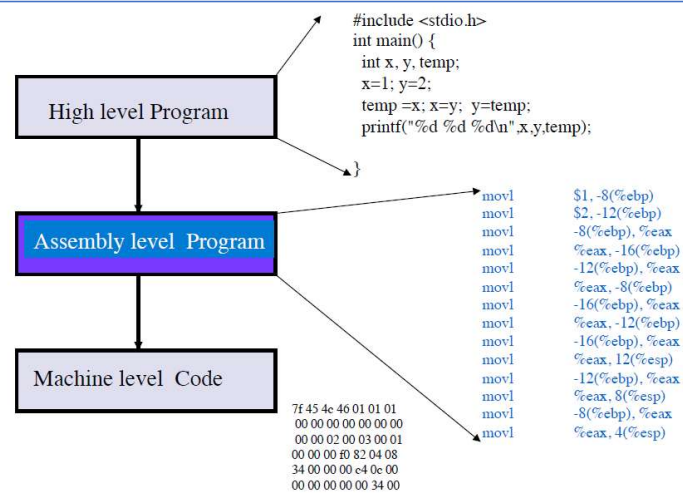
Spring 2026

Operating Systems

35

35

Languages to Hardware



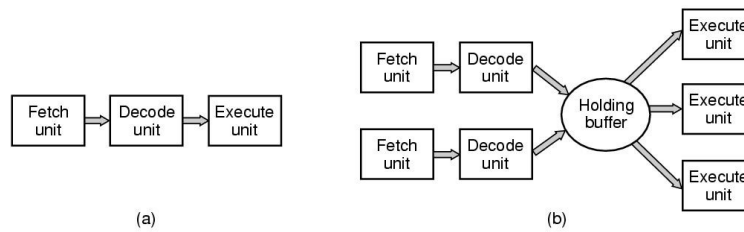
Spring 2026

Operating Systems

36

36

CPU Pipelining



(a) A three-stage pipeline.

(b) A superscalar CPU.

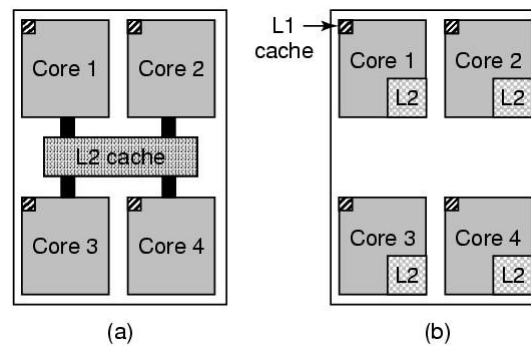
Spring 2026

Operating Systems

37

37

Multithreaded and Multicore Chips



(a) A quad-core chip with a shared L2 cache.

(b) A quad-core chip with separate L2 caches.

Spring 2026

Operating Systems

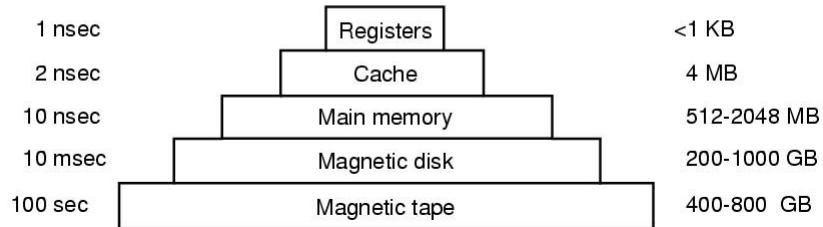
38

38

Memory (1)

Typical access time

Typical capacity



A typical memory hierarchy. The numbers are very rough approximations.

Spring 2026

Operating Systems

39

39

Memory (2)

- Questions when dealing with cache:
 - When to put a new item into the cache
 - Which cache line to put the new item in
 - Which item to remove from the cache when a slot is needed
 - Where to put a newly evicted item in the larger memory

Spring 2026

Operating Systems

40

40

Disks

- Load OS and applications from disk
- Manage disk space allocation
- Store and retrieve files by name
- Provide some protection for files

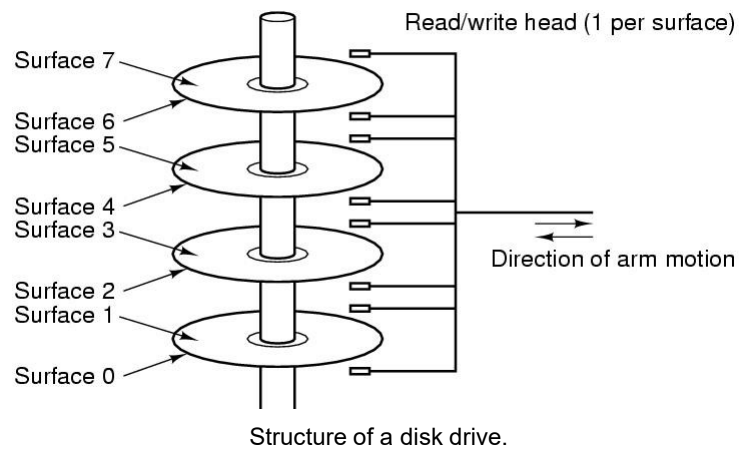
Spring 2026

Operating Systems

41

41

Disks



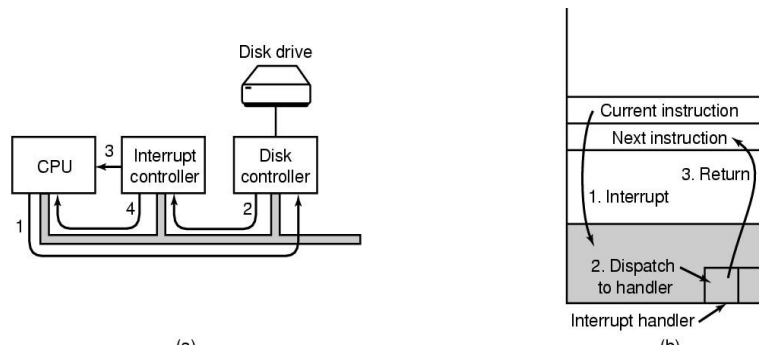
Spring 2026

Operating Systems

42

42

I/O Devices



(a) The steps in starting an I/O device and getting an interrupt.

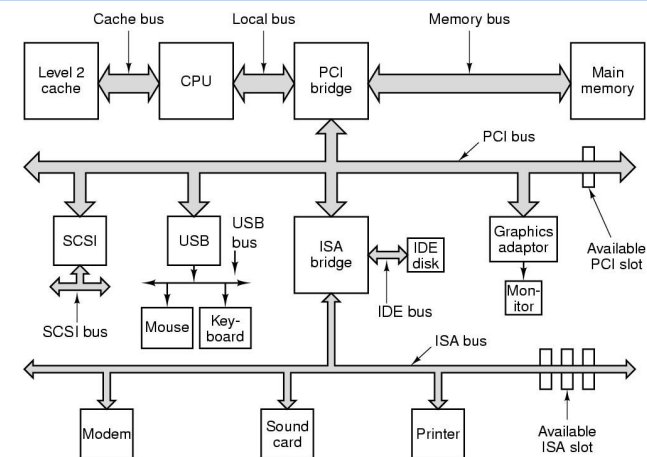
Spring 2026

Operating Systems

43

43

Buses



The structure of a large Pentium system

Spring 2026

Operating Systems

44

44

Timesharing

- Support people on low-speed terminals
- Systems ranged from remote editing and batch submission to specialized restricted environments (BASIC) to operating systems designed for the purpose
- Timesharing systems stressed responsiveness

Spring 2026

Operating Systems

45

45

Controlling Monitors and Early OSs

- Command language — moral equivalent of shell
- Supply accounting information
- Request resources — memory, CPU time, tapes, disk space
- Invoke user or system programs
- Batch-oriented — submit your “job” on punch cards, pick it up a few hours later

Spring 2026

Operating Systems

46

46

Enter the Minicomputer

- Improved technology permitted small, (relatively) cheap minicomputers (\$20-40K)
- May or may not have had disks; programmed by punch cards or paper tape
- No memory protection — ran on monitors, not operating systems
- Followed the same evolutionary path as mainframes
- Most important early mini: DEC PDP-8

Spring 2026

Operating Systems

47

47

High-Level Languages and OSs

- Early OSs written in assembler
- Notable exception: Burroughs 5000 designed for Algol (1961!)
- Multics (started in 1965) used PL/I
- Unix (early 1970s) used C
- In mid-1970s, IBM migrated to PL/S, a medium-level language

Spring 2026

Operating Systems

48

48

Microcomputers

- By the late 1970s, minicomputers had memory protection and real operating systems
- By then, microcomputers had come along
- These had no memory protection, no operating systems, and no disks. . .
- Windows XP is the first version of Windows that has few architectural concessions to this history
- But the habit of users running as Administrator dates back to that era
- Many Windows security problems are due to the repeat of 1950s computer history!

Spring 2026

Operating Systems

49

49

Modern Operating Systems

- Virtual memory, disk, multiprogramming
- Multi-CPU support common (and about to become ubiquitous)
- Designed for highly-interactive use (GUI)
- Network I/O crucial

Spring 2026

Operating Systems

50

50

Operating System Range

- Mainframes — vast I/O bandwidth (Unix, MVS, VM/CMS)
- Servers (Solaris, Linux, Windows 2003)
- Desktops — premium on interaction: mouse, graphics (Windows, Solaris, Linux)
- Embedded systems — often lack memory protection (VxWorks, Symbian)



Spring 2026

Operating Systems

51

51

Hardware Considerations

- What must the hardware do to accommodate a modern OS?
- Is there some reason, other than short-sightedness, why we've seen this endless repetition of design decisions?

Spring 2026

Operating Systems

52

52

Memory Protection

- Protect OS from being overwritten by user programs
- Prevent sensitive OS data from being read by users programs
- Protect user programs from reading or writing other programs' memory

Spring 2026

Operating Systems

53

53

Who Shall Watch the Watchmen?

- The ability to change memory protection must be unavailable to user programs
- In other words, the OS needs access to some parts of the hardware that application programs cannot access
- Two basic strategies: special (protected) memory areas and privileged instructions
- In practice, both are used

Spring 2026

Operating Systems

54

54

Special Memory Areas

- Writing to certain special memory addresses controls some aspects of the machine
- Example: memory protection can be turned on and off by writing to other memory areas
- These other areas are only accessible to the OS, using the exact same mechanism

Spring 2026

Operating Systems

55

55

Privileged Operations

- There exists a CPU state flag that allows or disallows certain operations
- When the OS is running, this flag is set to “privileged”
- The privileged flag can only be changed when running in privileged mode
- To run an application program, this flag is set to unprivileged

Spring 2026

Operating Systems

56

56

Returning to the OS

- When an application program is run, certain memory areas are inaccessible and the privilege flag is off
- By definition, the application cannot change this
- How do we return to the OS?
- Two mechanisms: interrupts and system calls

Spring 2026

Operating Systems

57

57

Interrupts

- External signal to the CPU
- Hardware saves the CPU state and loads new state
- New state includes privileged flag and access to OS memory
- Starts running OS's interrupt handler

Spring 2026

Operating Systems

58

58

I/O Interrupts

- Notify the CPU when an I/O operation is complete
- Vital to multiprogramming — let's the CPU run another job while I/O is taking place
- Eliminates need for busy wait

Spring 2026

Operating Systems

59

59

Other Interrupts

- Timer
- Button-push
- “Shoulder tap” from another CPU
- Environmental (power fail)

Spring 2026

Operating Systems

60

60

System Calls

- Initiated by application program
- Conceptually similar to a function call
- Again, the hardware saves the CPU state and loads new state
- Again, the new state includes privileged flag and access to OS memory
- Hardware starts executing OS's system call handler
- On some architectures, just another kind of interrupt

Spring 2026

Operating Systems

61

61

Other Traps

- Some traps happen due to program behavior
- Illegal operations, privileged operation, memory protection violation, divide-by-zero, etc.
- Virtual memory-related traps require OS assistance

Spring 2026

Operating Systems

62

62

Saving State

- What state does the hardware save?
- As little as possible — saving state is expensive
- Minimum: privileged status, memory access, return address (often combined in a Program Status Word (PSW))
- Anything else likely to be changed by early part of interrupt handle (i.e., branch condition code)
- Often collected into a processor status word

Spring 2026

Operating Systems

63

63

Where is State Saved?

- One obvious place is on the stack
- What stack?
- The OS can't trust the application's stack
- Conclusion: must save state in an OS-only location
- As we shall see, most of the state has to be saved under software control

Spring 2026

Operating Systems

64

64

Masking Interrupts

- The CPU can generally block interrupts temporarily
- The interrupt mask state is often part of the PSW
- PSW after an interrupt generally blocks new interrupts, to avoid recursion

Spring 2026

Operating Systems

65

65

Interrupts and Complexity

- Interrupts occur asynchronously to application programs
- In general, the interrupted program is not the one interested in the interrupt
- They are a major cause of OS complexity

Spring 2026

Operating Systems

66

66

The Operating System Zoo

- Mainframe operating systems
- Server operating systems
- Multiprocessor operating systems
- Personal computer operating systems
- Handheld operating systems
- Embedded operating systems
- Sensor node operating systems
- Real-time operating systems
- Smart card operating systems



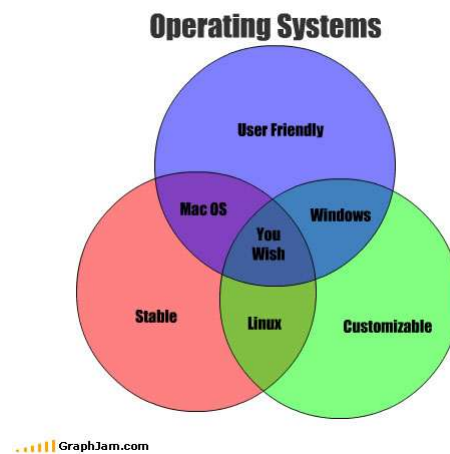
Spring 2026

Operating Systems

67

67

How to choose?



Spring 2026

Operating Systems

68

68

Operating System Concepts

- Processes
- Address spaces
- Files
- Input/output
- Protection
- The shell
- Ontogeny recapitulates phylogeny
 - Large memories
 - Protection hardware
 - Disks
 - Virtual memory

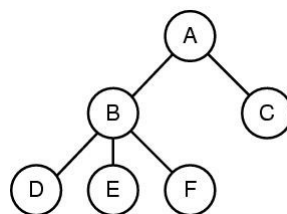
Spring 2026

Operating Systems

69

69

Processes



A process tree. Process A created two child processes, B and C. Process B created three child processes, D, E, and F

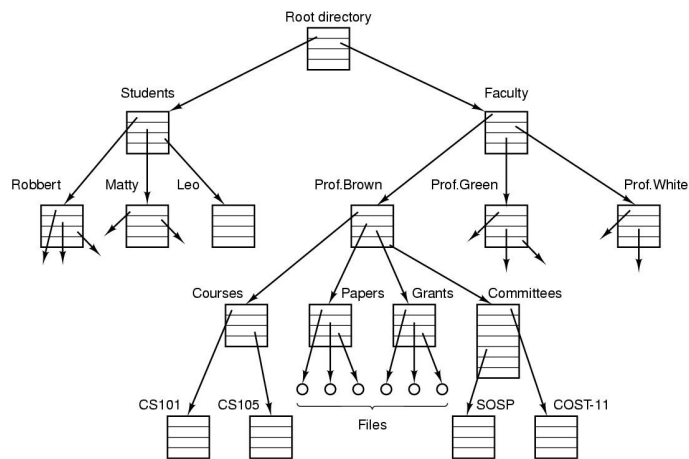
Spring 2026

Operating Systems

70

70

Files (1)



A file system for a university department

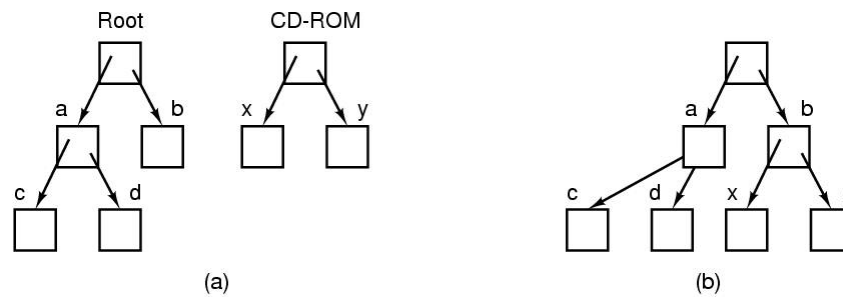
Spring 2026

Operating Systems

71

71

Files (2)



(a) Before mounting, the files on the CD-ROM are not accessible.

(b) After mounting, they are part of the file hierarchy

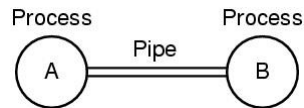
Spring 2026

Operating Systems

72

72

Files (3)



Two processes connected by a pipe

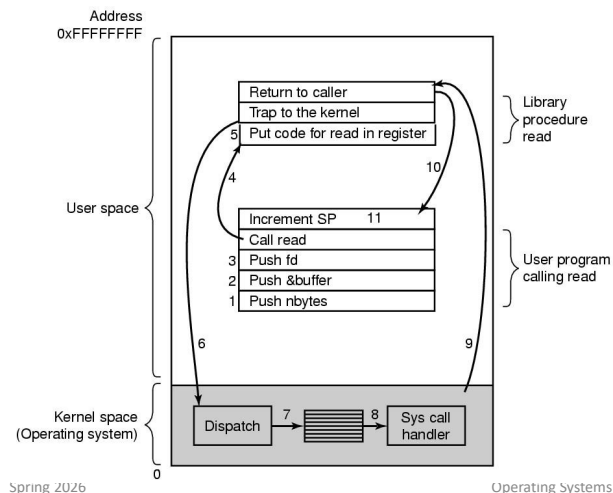
Spring 2026

Operating Systems

73

73

System Calls



The 11 steps in making the system call
`read(fd, buffer, nbytes)`

Spring 2026

Operating Systems

74

74

System Calls for Process Management

Process management

Call	Description
pid = fork()	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

Some of the major POSIX system calls

Spring 2026

Operating Systems

75

75

System Calls for File Management (1)

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing, or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Some of the major POSIX system calls

Spring 2026

Operating Systems

76

76

System Calls for File Management (2)

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Some of the major POSIX system calls

Spring 2026

Operating Systems

77

77

Miscellaneous System Calls

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970

Some of the major POSIX system calls

Spring 2026

Operating Systems

78

78

A Simple Shell

```

#define TRUE 1

while (TRUE) {                                /* repeat forever */
    type_prompt( );                          /* display prompt on the screen */
    read_command(command, parameters);       /* read input from terminal */

    if (fork( ) != 0) {                      /* fork off child process */
        /* Parent code. */
        waitpid(-1, &status, 0);            /* wait for child to exit */
    } else {
        /* Child code. */
        execve(command, parameters, 0);     /* execute command */
    }
}

```

A stripped-down shell

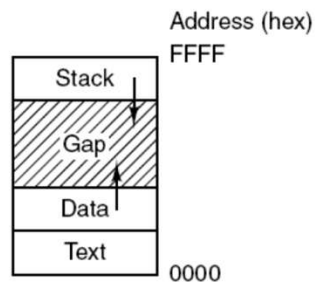
Spring 2026

Operating Systems

79

79

Memory Layout



Processes have three segments: text, data, and stack

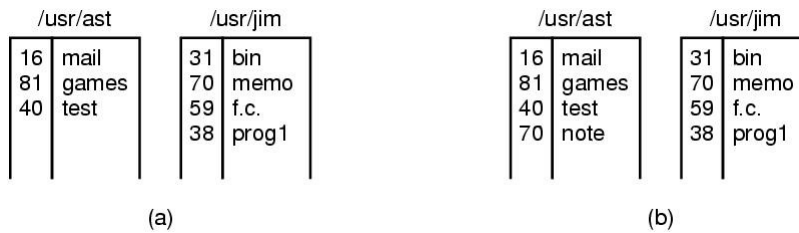
Spring 2026

Operating Systems

80

80

Linking



(a) Two directories before linking `/usr/jim/memo` to `ast's` directory

(b) The same directories after linking

Spring 2026

Operating Systems

81

81

Mounting



(a) File system before the mount

(b) File system after the mount

Spring 2026

Operating Systems

82

82

Windows Win32 API

UNIX	Win32	Description
fork	CreateProcess	Create a new process
waitpid	WaitForSingleObject	Can wait for a process to exit
execve	(none)	CreateProcess = fork + execve
exit	ExitProcess	Terminate execution
open	CreateFile	Create a file or open an existing file
close	CloseHandle	Close a file
read	ReadFile	Read data from a file
write	WriteFile	Write data to a file
lseek	SetFilePointer	Move the file pointer
stat	GetFileAttributesEx	Get various file attributes
mkdir	CreateDirectory	Create a new directory
rmdir	RemoveDirectory	Remove an empty directory
link	(none)	Win32 does not support links
unlink	DeleteFile	Destroy an existing file
mount	(none)	Win32 does not support mount
umount	(none)	Win32 does not support mount
chdir	SetCurrentDirectory	Change the current working directory
chmod	(none)	Win32 does not support security (although NT does)
kill	(none)	Win32 does not support signals
time	GetLocalTime	Get the current time

The Win32 API calls

Spring 2026

Operating Systems

83

83

Operating Systems Structure

- Monolithic systems – basic structure:
 - A main program that invokes the requested service procedure
 - A set of service procedures that carry out the system calls
 - A set of utility procedures that help the service procedures

Spring 2026

Operating Systems

84

84

Monolithic Systems

```
#define FALSE 0
#define TRUE 1
#define N      2          /* number of processes */

int turn;                  /* whose turn is it? */
int interested[N];         /* all values initially 0 (FALSE) */

void enter_region(int process); /* process is 0 or 1 */
{
    int other;              /* number of the other process */

    other = 1 - process;    /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;         /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

A simple structuring model for a monolithic system

Spring 2026

Operating Systems

85

85

Layered Systems

Layer	Function
5	The operator
4	User programs
3	Input/output management
2	Operator-process communication
1	Memory and drum management
0	Processor allocation and multiprogramming

Structure of the THE operating system

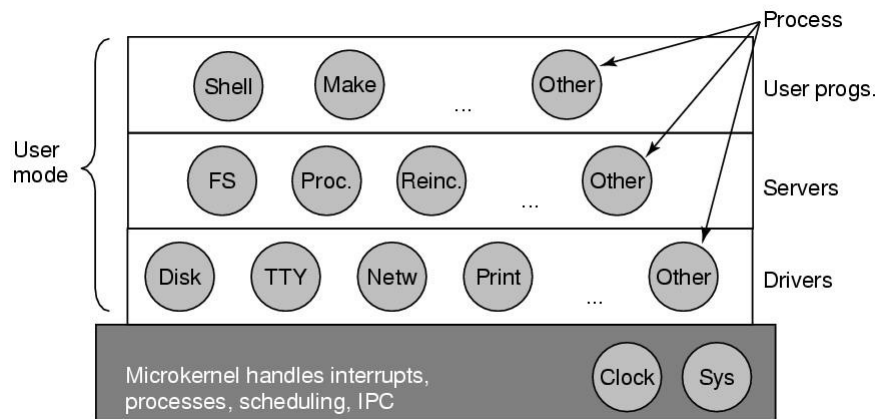
Spring 2026

Operating Systems

86

86

Microkernels



Structure of the MINIX 3 system

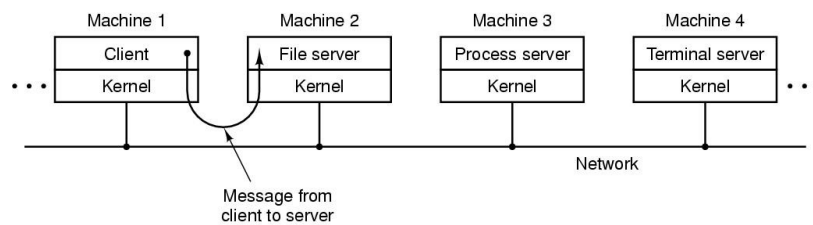
Spring 2026

Operating Systems

87

87

Client-Server Model



The client-server model over a network

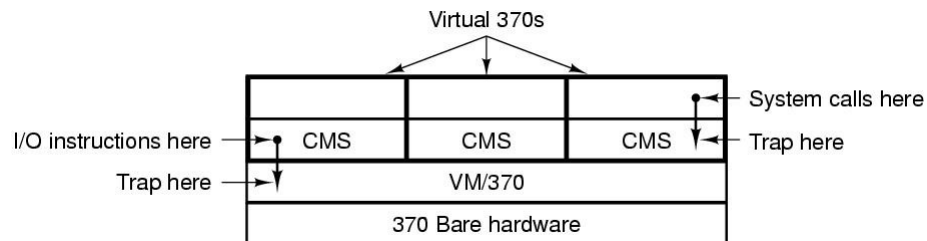
Spring 2026

Operating Systems

88

88

Virtual Machines (1)



The structure of VM/370 with CMS

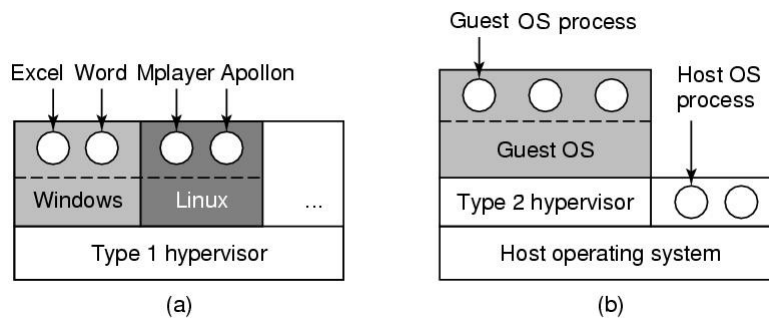
Spring 2026

Operating Systems

89

89

Virtual Machines (2)



(a) A type 1 hypervisor

(b) A type 2 hypervisor

Spring 2026

Operating Systems

90

90

Operating System Components

- What's in an OS?
- Is there a neat boundary?
- What clearly isn't part of the OS?

Spring 2026

Operating Systems

91

91

The Kernel

- Fundamental piece of the OS
- At a minimum, uses hardware protection mechanisms to isolate other pieces
- Provides communication paths linking different pieces of the system
- Provides interface to user application programs

Spring 2026

Operating Systems

92

92

Scheduler

- Allocates CPU time
- Handles scheduling priorities
- Ensures fairness

Spring 2026

Operating Systems

93

93

Memory Management

- Allocates memory to user applications
- Utilizes virtual memory to extend real memory space
- Ensures separation of memory between different applications

Spring 2026

Operating Systems

94

94

File Systems

- Layer files, directories, etc., on top of raw disk
- Enforce permissions

Spring 2026

Operating Systems

95

95

Device Drivers

- Hardware-dependent code to talk to I/O devices
- Provide type-specific common interface to higher layers
- Isolates hardware-specific complexity
- Part of the OS to prevent improper access to devices (other reasons as well)

Spring 2026

Operating Systems

96

96

More?

- Is the login command part of the OS?
- What about the mount command?
- The shell?
- The C compiler?
- A text-formatting package?
- Different people draw the line in different places
- One useful metric: Who can replace the component with a different one?

Spring 2026

Operating Systems

97

97

The World According to C

- The C language
- Header files
- Large programming projects
- The model of run time

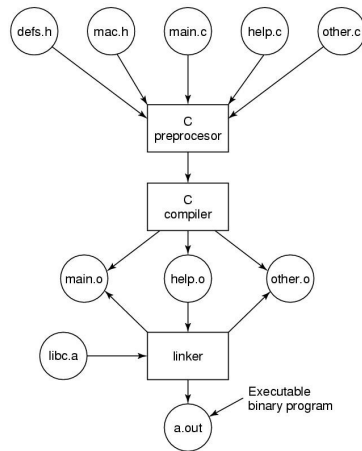
Spring 2026

Operating Systems

98

98

The Model of Run Time



The process of compiling C and header files to make an executable

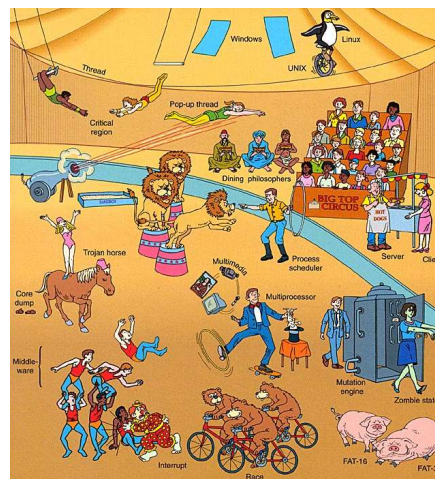
Spring 2026

Operating Systems

99

99

Orchestration



Spring 2026

Operating Systems

100

100

See you next time!